libbioestat - Biblioteca modular de Bioestatística em C

Equipe 22: Gustavo Yamani, Matheus Guimarães Olegario, Matheus Moreira Cabral e Thiago Assumpção Baisch





Domínio Escolhido e Problema

Por que Bioestatística?

- Dominio modular por natureza
- Entradas e saídas matematicamente bem definidas

Problema que Resolve

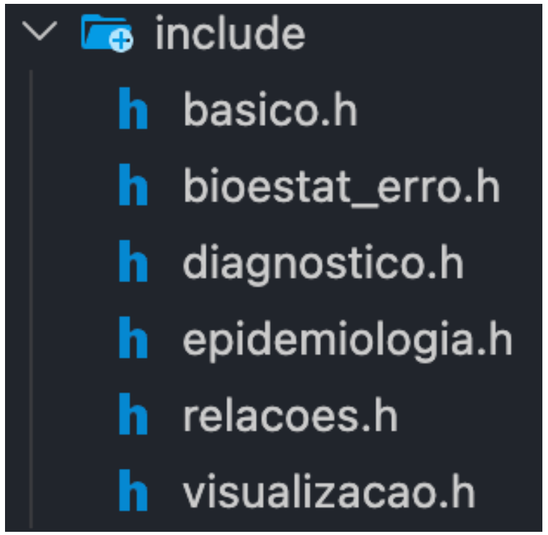
- Fornece cálculos rápidos e seguros de estatística descritiva, associação epidemiológica, testes diagnósticos e regressão , com tratamento rigoroso de erros

Entrada de dados

- Vetores de números reais (double *, size_t n)
- Tabela 2x2
- Números inteiros

Arquitetura e Módulos

- bioestat_erro: Convenção padronizada de códigos de retorno (be_status)
- basico: Estatística descritiva (média, mediana, quartis, variância, desvio padrão, moda)
- relacoes: Associação (covariância, correlação linear, regressão linear, previsão)
- diagnostico: Qualidade de testes (sensibilidade, especificidade, VPP, VPN)
- epidemiologia: Tabela 2x2 e riscos (RR, Odds Ratio, Diferença de Riscos)
- visualizacao: Renderização de gráficos em ASCII (terminal) e vetoriais (SVG/HTML)



Convenção de Erros

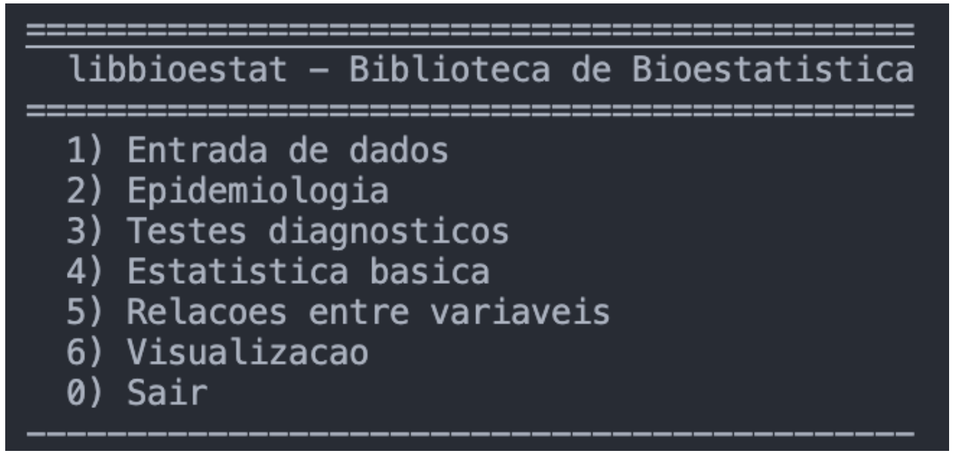
- O retorno de uma função da biblioteca sempre é um be_status
- O resultado do cálculo sempre é gravado em um ponteiro de saída
- Se algum erro ocorrer o valor original do ponteiro é preservado

```c
#include "../include/bioestat_erro.h"

const char *be_status_mensagem(be_status status)
{
    switch (status) {
    case BE_OK:                   return "sucesso";
    case BE_ERRO_PONTEIRO_NULO:   return "ponteiro nulo recebido";
    case BE_ERRO_VALOR_INVALIDO:  return "valor de entrada invalido";
    case BE_ERRO_DIVISAO_POR_ZERO: return "divisao por zero no calculo";
    case BE_ERRO_MEMORIA:         return "erro de memória";
    case BE_ERRO_NAO_IMPLEMENTADO: return "funcao ainda nao implementada";
    }
    return "erro desconhecido";
}
```

Aplicação CLI

- Menu em terminal
- Erros mostrados via be_status direto da biblioteca
- Modo --demo roda bateria com dados pré-configurados sem aguardar teclado
- Gráficos em ASCII no próprio terminal (make run)



Makefile

- Descoberta automática de fontes com wildcard e patsubst
- Regras .PHONY: all, run, test, install, uninstall, clean, doc.
- Compilação incremental (-MMD -MP): O GCC gera arquivos .d para mapear alterações em arquivos .h

```
# Todos os .c de src/ viram .o em build/
LIB_SRCS = $(wildcard src/*.c)
LIB_OBJS = $(patsubst src/%.c,build/%.o,$(LIB_SRCS))
# Todos os .c de app/ viram .o em build/app/ (a aplicação tem vár
APP_SRCS = $(wildcard app/*.c)
APP_OBJS = $(patsubst app/%.c,build/app/%.o,$(APP_SRCS))

STATIC_LIB = lib/libbioestat.a
SHARED_LIB = lib/libbioestat.$(SHARED_EXT)

# Todo arquivo tests/test_*.c vira um executavel de teste em bin/
TEST_SRCS = $(wildcard tests/test_*.c)
TEST_OBJS = $(patsubst tests/%.c,build/tests/%.o,$(TEST_SRCS))
TEST_BINS = $(patsubst tests/%.c,bin/tests/%,$(TEST_SRCS))
```

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c11 -Iinclude
```

```
DEPFLAGS = -MMD -MP
```

```
build/%.o: src/%.c
    mkdir -p build
    $(CC) $(CFLAGS) $(DEPFLAGS) -fPIC -c $< -o $@
```

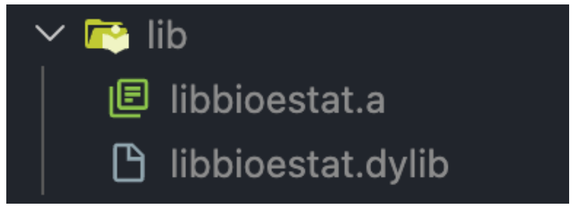
Biblioteca estática x dinâmica

Geração Paralela:

- libbioestat.a → Empacotada com ar rcs. Código fonte embutido no binário final
- libbioestat.so → Compilada com -fPIC e -shared, ligada em tempo de execução via rpath

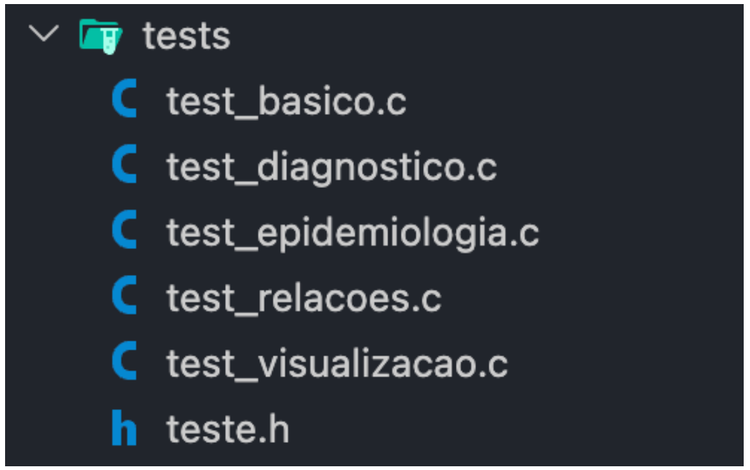
```
# Biblioteca estatica aqui: é um "pacote" com
# Alvo: a biblioteca. Pré requisitos: LIB_OBJS
$(STATIC_LIB): $(LIB_OBJS)
    mkdir -p lib
    ar rcs $@ $^

# Biblioteca dinamica: os mesmos .o, ligados r
$(SHARED_LIB): $(LIB_OBJS)
    mkdir -p lib
    $(CC) $(SHARED_FLAGS) -o $@ $^ $(LDLIBS)
```



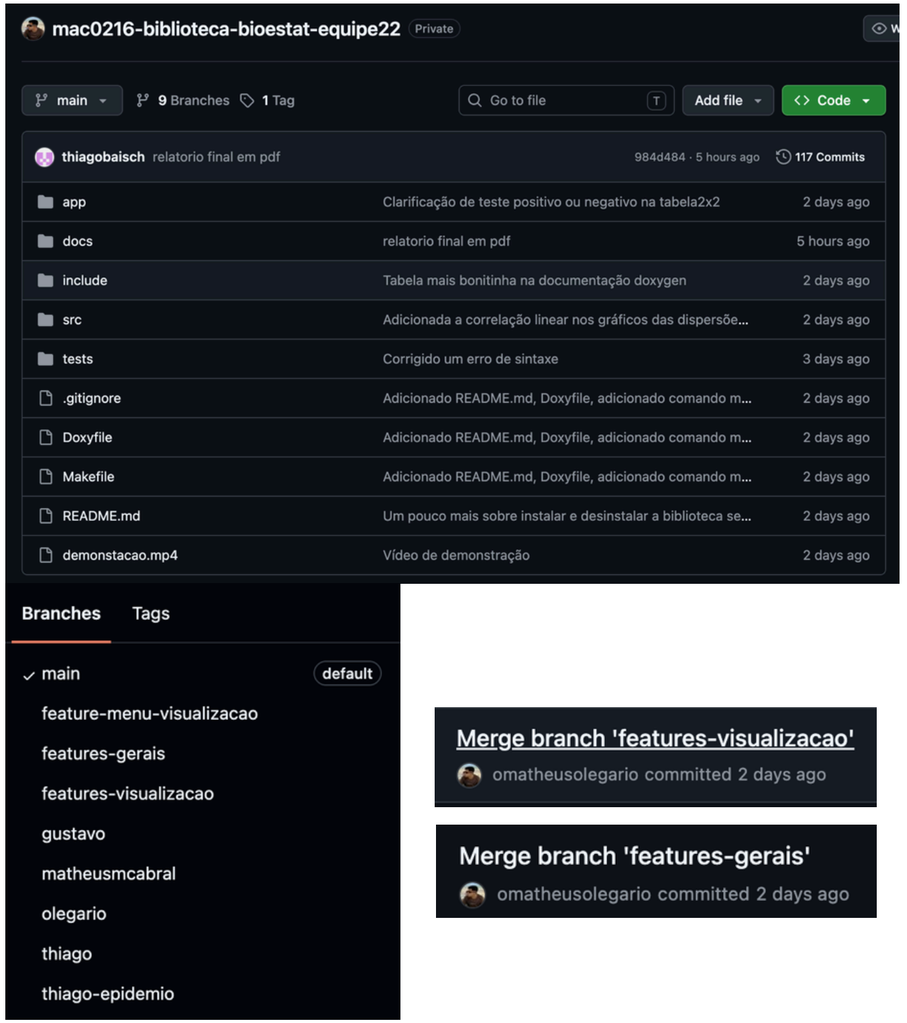
Estratégia de testes e Caso Prático de Falha

- Mini framework de testes em C (tests/teste.h)
- 291+ verificações cobrindo casos normais, de borda, ponteiros nulos e divisões por zero
- Caso Prático de falha:
  - be_odds_ratio, (axd)/(bxc) com contagens long grandes estourava antes da conversão para double.
  - No teste Tabela {4000000000, 1, 1, 4000000000} estourava o limite.
  - A solução foi um cast explícito para double antes de cada multiplicação, zerando os problemas.



Versionamento e Git

- 113 commits e 8 branches durante o desenvolvimento
- v1.0.0 marcando o estado final da entrega
- Cada módulo foi isolado em sua branch
- Integração na main via merges e pull requests após aprovação nos testes
- .gitignore configurado para evitar envio de binários, objetos, dependências e documentação doxygen



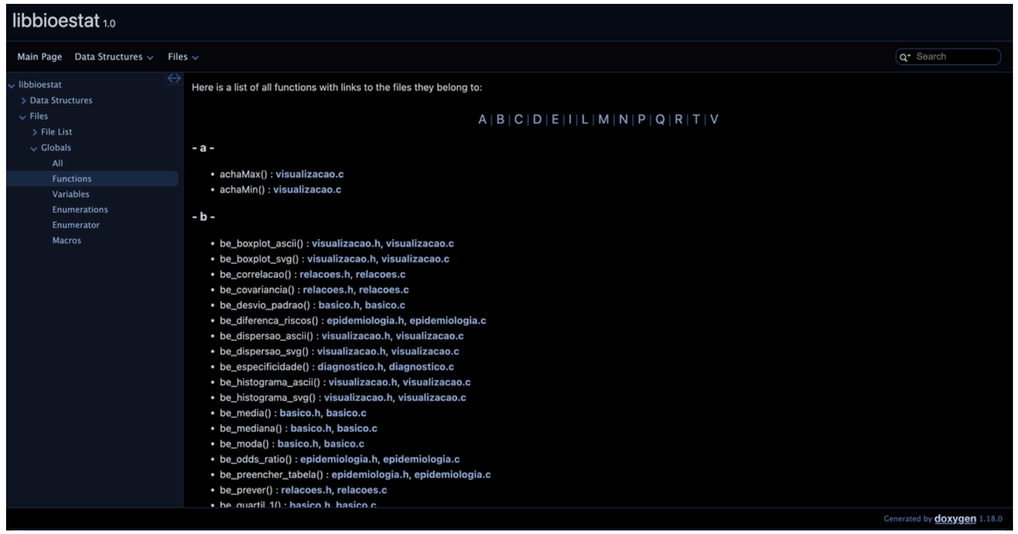
Dificuldades encontradas

- Padronização tardia da API com relação a nomenclatura e erros
- Compatibilidade do makefile entre linux e macOS (.so vs .dylib e flags de rpath)
- Evitar leaks na geração de gráficos ASCII/SVG e no gerenciamento da memória heap da aplicação.

```
UNAME_S := $(shell uname -s)
ifeq ($(UNAME_S),Darwin)
    SHARED_EXT   = dylib
    SHARED_FLAGS = -dynamiclib -install_name @rpath/libbioestat.dylib
    RPATH        = -Wl,-rpath,@executable_path/../lib
else
    SHARED_EXT   = so
    SHARED_FLAGS = -shared
    RPATH        = -Wl,-rpath,'$$ORIGIN/../lib'
endif
```

Conclusão

- 29 funções públicas e 100% testadas em 5 módulos de bioestatística
- Makefile completo com compilação estática/dinâmica, suporte a install/uninstall e documentação Doxygen (make doc)



Perguntas e Demonstração (make run)